

Prompt to Response Web App

Project Details

Role: Software Developer

Project Type: fullstack

Difficulty: junior

Duration: 1 day

Focus: fullstack

Student Name: Rahul Sharma

Description: Build a simple web app that lets users submit a prompt, forwards it to the Groq API via Kafka, stores the conversation in MySQL, and displays the response.

Technologies

- React
- Python
- REST API
- Groq API
- Kafka
- MySQL

Requirements

- Create a prompt form on the frontend.
- Send the prompt to a Python REST endpoint.
- Publish the prompt to a Kafka topic.
- Consume the topic, call Groq API, store and return the response.
- Persist conversation data in MySQL.

Objectives

- Implement a working end to end flow.
- Demonstrate API design and message queue usage.
- Show basic persistence and data retrieval.

Functional Requirements

- FR1: Prompt form must POST to /api/prompt.
- FR2: Backend must publish prompt to Kafka topic "prompts".
- FR3: Consumer must call Groq API and store response.
- FR4: Response must be returned to frontend via /api/response.
- FR5: Conversation must be stored in MySQL with user, prompt, and reply.

Non Functional Requirements

- NFR1: Input must be validated and sanitized.
- NFR2: API responses must use JSON and proper status codes.
- NFR3: Kafka messages must be idempotent.
- NFR4: MySQL schema must enforce foreign keys.

Modules

Frontend

Description: React UI for prompt entry and response display

Features

- Prompt form
- Response viewer
- API integration

Backend

Description: Python REST API with Kafka producer/consumer and MySQL persistence

Features

- Prompt endpoint
- Response endpoint
- Kafka integration

Implementation Plan

Setup

Tasks

- Initialize React app
- Create FastAPI project
- Set up MySQL schema
- Configure Kafka broker

Development

Tasks

- Build prompt form
- Implement /api/prompt endpoint
- Publish to Kafka
- Create consumer to call Groq API
- Persist data
- Return response

Testing & Deployment

Tasks

- Unit tests for API
- Integration test with Kafka
- Verify MySQL persistence
- Run end to end demo

Evaluation Criteria

Correctness

Description: All functional requirements are met and pass tests

Weight: 25

Core Functionality

Description: End to end flow works as intended

Weight: 20

Problem Solving

Description: Efficient use of Kafka and API calls

Weight: 15

Code Quality

Description: Readable, modular code with comments

Weight: 15

Architecture

Description: Clear separation of concerns

Weight: 10

Testing

Description: Comprehensive test coverage

Weight: 10

Documentation

Description: README and API docs present

Weight: 5

Deliverables

- Source code repository
- README with setup steps
- API documentation (OpenAPI)
- MySQL schema diagram
- Unit test suite
- Demo video or GIF

Database Design

Item 1

Entity: User

Fields

- id PK
- name
- email

Item 2

Entity: Conversation

Fields

- id PK
- user_id FK
- created_at

Item 3

Entity: Message

Fields

- id PK
- conversation_id FK
- role
- content
- created_at

Testing Plan

- [object Object]
- [object Object]
- [object Object]
- [object Object]
- [object Object]

Student Id: 6ab2576039d4e38828d96a4e

Status: active